rdf-solve: Interoperable RDF for the Life Sciences through Automated Schema Mining and Semantic Mappings

Javier Millan Acosta Egon Willighagen More

March 10, 2026

Abstract

The life sciences community has produced many RDF knowledge bases encoding decades of curated biological data, collectively forming the Life Sciences Linked Open Data (LSLOD) cloud. RDF allows domains such as chemistry, agriculture, and disease research to define or reuse vocabularies, properties, and class hierarchies as schemas for their typed statements, but this flexibility fragments representation: for instance, the same aminoacid sequence may be modeled differently across datasets depending on whether it is treated as a sequence, a structure, or a pathway component. Schemas are usually documented in publications rather than as linked data, so finding which classes and properties exist across resources remains largely manual and must be repeated when databases are updated. Prior meta-analyses estimated vocabulary reuse at under 20 % and concluded the cloud is less connected than assumed, yet their analyses remain descriptive: latent overlap is identified statistically but no integration bridges are constructed and no machine-readable schema artefacts are produced.

We describe a three-stage methodology. First, co-occurrence patterns are mined automatically from SPARQL endpoints to approximate database schemas, against locally deployed endpoints when data dumps are available or against remote endpoints directly, and serialized in widely adopted formats with full provenance metadata. Second, a mapping layer is constructed from instance-based URI probing and curated mapping sets, linking classes and properties across resources. Third, inference over the combined mapping graph materializes bridges via transitivity and inversion, yielding a connectivity graph that makes explicit which dataset pairs share compatible schema elements and can be concretely joined.

The methodology is implemented in rdf-solve and applied to N LSLOD datasets, recovering implicit schema structure and publishing it as machine-readable linked data and graph and relational exports, lowering the barrier for federated querying, data integration, and longitudinal schema monitoring.

1 Introduction

1.1 Knowledge representation in the life sciences

Knowledge bases (...). [?]

The Resource Description Framework (RDF) models information as subject–predicate–object triples whose nodes are identified by Uniform Resource Identifiers (URIs) [?]. RDF Schema (RDFS) and the Web Ontology Language (OWL) layer class hierarchies, property constraints, and logical definitions on top of this model [?]. The biomedical community was among the earliest adopters of Semantic Web technologies for knowledge management and clinical decision support [?], and the resulting Life Sciences Linked Open Data (LSLOD) cloud now spans RDF resources [?].

RDF offers several structural advantages for large-scale data integration. Its uniform syntax and schema-less flexibility support federated querying across multiple heterogeneous sources with minimal syntactic adaptation [?]. The use of URIs also makes resources web-dereferenceable: each entity can be looked up via standard HTTP [?]. Semantics encoded in RDFS and OWL enable software reasoners to verify consistency and generate novel inferences from existing assertions [?], and the linking model facilitates discovery of related resources even when underlying schemas differ [?].

1.2 The gap

Despite these properties, the LSLOD cloud faces significant interoperability challenges. There is semantic heterogeneity due to varying entity notations, diverse schemas, and a insufficient common vocabulary reuse [?]. Analyzing complex knowledge graphs requires extensive computational skills and deep knowledge of domain-specific terminologies [?], and researchers often struggle to find sources available and accessible on the Web [?]. Recent work on zero-shot SPARQL generation shows that even LLM agents must probe endpoint schemas interactively before they can construct correct queries [?], which points at the practical importance of discoverable, machine-readable schema information. Neural

machine translation approaches to SPARQL face a related bottleneck: generating correct queries for rare or unseen URIs (the out-of-vocabulary problem) remains difficult without a machine-readable schema to ground generation [?].

Knowing which ontologies a dataset imports does not reveal its actual structure: which classes are instantiated, which entailments appear, how often a property appears, or where values are missing cannot be inferred from ontology declarations alone.

This is why wo datasets that (partially) reuse the same ontology can have very different practical shapes while two datasets with disjoint vocabularies can have overlapping patterns that remain invisible without mappings. A meta-analysis of approximately 80 LSLOD sources estimated vocabulary reuse at under 20 % and concluded that the cloud "is not actually as densely connected as is often indicated" [?].

A further conceptual barrier stems from the evolution of schema philosophies. The Semantic Web community has traditionally treated schemas as *descriptive*; promoting flexibility; while the database community regards them as *prescriptive*; enforcing validation [?]. The development of validation languages such as SHACL and ShEx [?, ?] reflects a convergence of these views.

RDF datasets attempt to reuse standard terms but often represent them with different notations or namespaces; e.g., varying URI patterns for the same ChEBI identifier; which technically prevents automated federation and integration[?, ?].

[?] read maillot.

1.3 Related work

Linked data profiling and RDF shape induction. Several profiling systems extract structural summaries from SPARQL endpoints. Loupe [?] mines class, property, and triple-pattern statistics and presents them through an interactive web interface. ABSTAT [?] computes ontology-driven linked data summaries with pattern minimization. ProLOD [?] profiles linked open data using statistical and structural analysis.

Mihindukulasooriya et al. [?] use machine learning; specifically Random Forest classifiers trained on profiled RDF data from Loupe; to automatically induce integrity constraints in SHACL and ShEx, achieving up to 97 % precision in cardinality estimation. Several methods have been developed to analyze and characterize RDF data, each addressing a distinct slice of the schema discoverability problem. IndeGx [?] VoID ext (find paper).

Tools to extract schemas automatically exist, including sheXer [?], VoID-generator[?],

Loupe [?], [?], but adoption among life sciences dataset providers is slow. Automated extraction is also technically constrained by endpoint timeouts and deeply nested constraint patterns. VoID-ext service descriptions, where published, can enable features like autocompletion [?], demonstrating the practical value of machine-readable schema metadata. QLever extends this idea engine-side: it precomputes per-subject predicate patterns at index time to deliver context-sensitive SPARQL autocompletion with faster responses even over very large graphs [?], and more recently supports zero-shot SPARQL generation from natural language via an LLM agent that probes the endpoint schema interactively [?]. Still, most workflows, providers and endpoints do not explicitly share schema artifacts for downstream, decentralized uses.

Others... decide title Other frameworks to describe the contents of databases (...) such as The DBCLS-developed rdf-config [?], which provides a human-readable YAML schema representation, manually curated for resources in the RDF Portal and used for finding links across the databases hosted in rdf Portal. SPARQL query collections offer another useful complement for describing RDF resources, illustratign possible uses for a database. More recently [?] describes reusing these queries as modules to allow users to dig deeper (...).

Exploration and Analytics of RDF. Sparklis [?] allows users to build analytical queries using natural language guidance directly against SPARQL endpoints. SynopsViz [?] provides hierarchical visual exploration and multilevel charting of large RDF datasets through incremental data retrieval and hierarchical aggregation [?]. Achieving high performance for complex aggregate queries on large knowledge graphs remains an open challenge [?]. shexer [?]; and shex-consolidator [?] for merging shape schemas. Sparqloscope [?] generates targeted benchmarks from dataset statistics to identify engine-specific strengths and weaknesses across SPARQL 1.1 features, revealing for example that block-indexed engines handle `GROUP BY` with many groups far better than hash-based alternatives at billion-triple scale. Dense retrieval approaches adapt BERT-based re-ranking to RDF search by linearizing representative triples as model input [?], but scaling such methods to heterogeneous LSLOD endpoints remains an open problem.

Federated query planning and decentralized data access. Prior literature work reveals knowing *which endpoints hold which data* is a prerequisite for efficient federated execution regardless of the underlying access model. Federated SPARQL execution requires source selection: determining which federation members hold data relevant to each

subquery. FedQPL [?] formalizes this as logical query plans over heterogeneous RDF interfaces. Link Traversal Query Processing (LTQP) [?] addresses this by using structural assumptions (Type Indexes) to prune the traversal space.

LSLOD meta-analysis. Most directly related to the present work, [?] conducted a comprehensive meta-analysis of the LSLOD cloud. Their Extraction Algorithm enumerates classes and properties connecting them for a given endpoint endpoint. The extracted schema graph was used to analyze vocabulary reuse (estimated at 19.86 %), detect communities of similar content via GloVe word-embedding similarity and Louvain clustering, and quantify inter- and intra-linking patterns. Their finding that the LSLOD cloud is not actually as densely connected as is often indicated directly motivates the present work. However, their approach remains diagnostic: it identifies latent overlap via community detection but does not materialize integration bridges or produce actionable schema artefacts (other than the pickle NetworkX archived in figshare). The authors call for top-down approaches to monitor source accessibility and enforce vocabulary reuse standards [?].

2 Methods

2.1 Scope

Formally, schema-based interoperability requires two capabilities. The first is measuring *consistency* within a dataset: how stable is a structural pattern across instances? Consistency scores, such as the coverage annotations introduced in sheXer, capture how often a class–property–value-type pattern actually occurs. The second is measuring *alignment* across datasets: identifying classes or properties in different graphs that correspond to each other. This requires curated mappings or cross-references that link entities across resources.

Several tools address parts of this space in isolation, however, no existing system connects schema extraction, multi-format serialization, mapping inference, and cross-resource connectivity analysis to effectively generate queries into a single pipeline.

rdf-solve addresses this gap. Building on the class-then-property paradigm of prior profiling work, it contributes:

1. When possible, local mining on QLever, which demands disk and RAM (Section **??**). Because a local QLever server imposes no network timeouts or rate limits, the pipeline

first attempts a *one-shot* strategy: a single unpaginated `SELECT DISTINCT` query that exhausts the full local index in one request. The multi-level fallback chain (bisection, adaptive pagination, predicate-first decomposition) is activated only if that attempt fails. Remote endpoints, where no data dump is available, always use the fallback chain;

2. Possibility to query remote endpoints with a fault-tolerant mining algorithm with multi-level fallbacks (bisection, adaptive pagination, predicate-first decomposition) that attempts to exhaust all possibilities before having to silently discard classes on failure (Section **??**);

3. a configurable treatment of untyped URI patterns: the default strategy records them as bare predicates, while an alternative strategy (`-untyped-as-classes`) promotes the object URIs to candidate classes, enabling a second pass over the same indexed data that reflects a different modelling assumption (Section **??**);

4. a mapping inference layer that materializes cross-endpoint integration bridges from entity-level mappings, curated mapping sets, and transitivity/inversion reasoning (Section **??**);

5. serialization of every extraction step in standard linked data formats (JSON-LD, VoID, SHACL, LinkML) with full provenance metadata, including author attribution, timing, and tool version, so that each artefact is independently citable; and

6. a connectivity graph that identifies which endpoint pairs can be concretely joined via shared ontology classes and inferred mappings (Section **??**).

The tool is open-source, CLI-driven, and ships as a containerized web application.

2.2 Schema Mining

rdf-solve approximates the schema of a SPARQL endpoint by mining co-occurrence patterns from instance data. The approach follows the class-then-property extraction paradigm introduced by linked data profiling tools [**?**, **?**] and used outside SPARQL [**?**] but extends it with three key differences: (i) configurable class batching via SPARQL `VALUES` bindings, reducing round-trips; (ii) a recursive fallback chain (bisection, pagination with adaptive

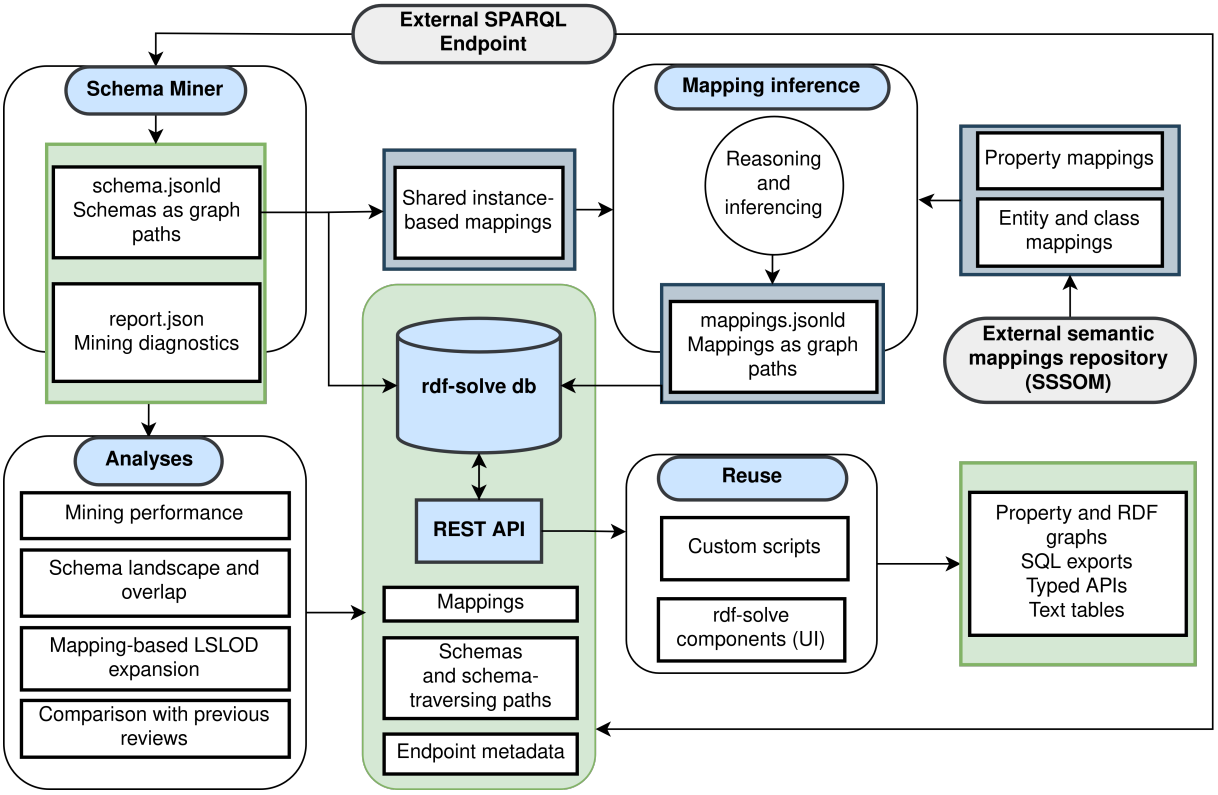

Figure 1: Workflow for the rdf-solve (...)

page sizing, and predicate-first decomposition) that degrades gracefully instead of discarding classes on failure; and (iii) separation of pattern discovery from counting, ensuring that a count-query failure does not lose already-discovered patterns.

Three pattern types are recorded: typed-object patterns (c, p, o) where o is the `rdf:type` of the object URI; literal patterns (c, p, dt) where dt is the object datatype; and untyped-URI patterns (c, p) where no `rdf:type` was retrievable for the object under the current graph scope and inference presented by the endpoint.

The treatment of the third pattern type is configurable. In the default strategy, untyped-URI patterns are retained as bare predicate links whose object type is unknown. An alternative strategy promotes those object URIs to candidate classes, effectively treating the absence of an explicit `rdf:type` assertion as a modelling gap rather than an absence of class membership. Both strategies can be applied to the same indexed data in consecutive passes, yielding two schema variants for the same source that can be compared or combined. This is particularly relevant for local mining (Section **??**), where re-running the algorithm over an already-indexed QLever instance adds negligible overhead.

Class and property labels are resolved from a priority-ordered label vocabulary: `rdfs:label` is preferred, followed by `skos:prefLabel`, `dc:title`, `IAO_0000118` (alternative term), and `skos:altLabel`. This order reflects decreasing terminological precision: primary semantic labels are preferred over bibliographic titles and curated synonyms. Falling back to the local URI fragment is a last resort. The multi-vocabulary lookup is relevant for endpoints that use OBO-style annotation properties rather than RDFS labels.

Notation. $Q(B, G)$ denotes query template Q evaluated for class batch B restricted to the named graphs in G, a set of zero or more URIs supplied via `-graph-uri`. When $G = \varnothing$ no `GRAPH` clause is added and the query runs over the endpoint's default dataset. When $|G| = 1$ a single `GRAPH <uri> { ...}` clause wraps the pattern; when $|G| > 1$ a `VALUES (?g) { ...} GRAPH ?g { ...}` block iterates over each named graph. In all cases the scope is a query-level restriction; endpoint dataset configuration may override or ignore it. $\top$ / $\bot$ denote success / failure (timeout or cost-limit). $B_{[:m]}$ and $B_{[m:]}$ are the two halves of B after splitting at $m = \lfloor |B|/2 \rfloor$; bisecting reduces the number of classes in the `VALUES` binding, narrowing the class-filter scope and isolating expensive classes into separate sub-queries. $\ell_{\min}$: minimum page size (configurable via `-chunk-size`) below which pagination is abandoned. Q_C: `SELECT DISTINCT ?c WHERE {?s a ?c}` (type enumeration). $Q_p(c)$: `SELECT DISTINCT ?p WHERE {?s a <c>; ?p ?o}` (predicate enu-

meration for class c). $Q^p_{\text{to}}(c,p)$: Q_{to} restricted to predicate p for class c. $t \models (s,p,o)$: triple $t = (s',p',o')$ matches pattern (s,p,o) operationally if the endpoint's evaluation of `?s a <s>; <p> ?o; ?o a <o>` (or the literal/datatype variant) binds t; this depends on the endpoint's graph scope, asserted `rdf:type` triples, and any active inference regime. $n_{(s,p,o)} = |\{t \mid t \models (s,p,o)\}|$ is the co-occurrence count as returned by the endpoint's `COUNT(*)` aggregate. Q^{cnt}_{to}, Q^{cnt}_{lit}, Q^{cnt}_{uri}: the corresponding `SELECT ...COUNT(*) ...GROUP BY` variants of Q_{to}, Q_{lit}, Q_{uri}; they accept the same `VALUES` batching and fallback chain.

Procedure. $\textsc{Query}(B,Q,E,G)$ sends $Q(B,G)$ to endpoint E as a single `SELECT DISTINCT` query. On $\top$ it returns the results immediately. On $\bot$, if $|B| > 1$ the batch is bisected and each half retried recursively: reducing the `VALUES` binding narrows the set of classes evaluated per sub-query, so that a class whose query exceeds the endpoint's cost limit does not cause the remaining classes in the same batch to be skipped. Pagination (`LIMIT`/`OFFSET`) is not tried for multi-class batches because it may not reduce per-page cost on endpoints that re-evaluate the full binding set for each page. For single-class batches ($|B| = 1$) the class filter is already minimal, so the fallback is pagination (`LIMIT` ℓ / `OFFSET` $k\ell$, $k = 0,1,2,\ldots$). If a page fails, ℓ is halved and that page is retried; if ℓ falls below $\ell_{\min}$ pagination is abandoned and, for Q_{to}, a predicate-first decomposition is attempted: $Q_p(B[0])$ enumerates predicates P and $Q^p_{\text{to}}(B[0],p)$ is called once per $p \in P$; if Q_p itself fails, the class contributes no typed-object patterns. Any results accumulated before pagination was abandoned are retained. If all strategies fail, $\textsc{Query}$ returns $\emptyset$.

Phase 1 enumerates the asserted instantiated types C visible under G. When a `class_chunk_size` is configured, queries are paginated with (`LIMIT`/`OFFSET`) steps; otherwise Q_C is sent as a single unpaginated query. In both cases the request is subject to the same bisect/paginate fallback chain as Phase 2. Phase 2 partitions C into batches B of configurable size and calls $\textsc{Query}$ three times per batch (Q_{to}, Q_{lit}, and Q_{uri}) accumulating patterns into S. If $G \neq \varnothing$ and Phase 2 yields $S = \emptyset$ despite $C \neq \emptyset$, an *ontology-graph fallback* is triggered: Phase 2 is re-run with $G' = \varnothing$, because the named graph may contain only class definitions while instances reside in the default graph. Optionally (enabled by default, disabled with `-no-counts`), a counting phase follows: the unique subject classes in S are re-batched identically to Phase 2 and each batch is submitted as a grouped `SELECT ...COUNT(*) ...GROUP BY` query through $\textsc{Query}$, so count queries benefit from the same bisect and pagination fallbacks as pattern queries. If a count query fails for a particular class (after all fallbacks), the corresponding patterns are retained in S with no count.

Execution strategy selection and schema completeness. For local QLever endpoints the preferred execution path is the one-shot strategy: a single unpaginated `SELECT DISTINCT` issued before any fallback is attempted. If it succeeds, the schema is complete in one round-trip and the fallback chain is never entered. Only when the one-shot query fails does the pipeline activate the multi-level fallback chain (Algorithm **??**). Remote endpoints always begin with the fallback chain, as the endpoint's timeout and cost-limit behaviour is unknown.

When a source has been mined under multiple execution strategies (e.g. default and `untyped-as-classes`, or local and remote), the pipeline selects the most complete result for downstream analysis. Completeness is assessed by two criteria applied in order: first, whether the counting phase succeeded for all classes (i.e. all patterns carry co-occurrence counts rather than null); second, total pattern count. The run with full count coverage is preferred; among runs with equal coverage, the one with the larger pattern set is retained. This selection is recorded in the provenance block of the emitted schema, allowing the choice to be audited and overridden.

Algorithm **??** summarizes the full procedure.

Algorithm 1 Schema pattern mining

Require: endpoint E; graph scope G
Ensure: S (observed co-occurrence patterns); $\emptyset$ for a batch may mean failure or genuinely no patterns; incompleteness is logged but not encoded in S

$S \leftarrow \emptyset$
Phase 1 — class discovery
$C \leftarrow \textsc{Query}(\varnothing, Q_C, E, G)$ ▷ paginated if `class_chunk_size` set
Phase 2 — pattern mining
for each batch $B \subseteq C$ **do**
  $S \leftarrow S \cup \textsc{Query}(B, Q_{\text{to}}, E, G)$ ▷ typed-object patterns
  $S \leftarrow S \cup \textsc{Query}(B, Q_{\text{lit}}, E, G)$ ▷ literal patterns
  $S \leftarrow S \cup \textsc{Query}(B, Q_{\text{uri}}, E, G)$ ▷ untyped-URI patterns
end for
if $S = \emptyset$ **and** $C \neq \emptyset$ **and** $G \neq \varnothing$ **then** ▷ ontology-graph fallback
  re-run Phase 2 with $G' = \varnothing$
end if
Counting phase (optional; skipped with `-no-counts`)
$N \leftarrow \emptyset$
for each batch $B \subseteq C_S$ **do** ▷ C_S: unique subject classes in S
  $N \leftarrow N \cup \textsc{Query}(B, Q^{cnt}_{\text{to}}, E, G)$ ▷ typed-object counts
  $N \leftarrow N \cup \textsc{Query}(B, Q^{cnt}_{\text{lit}}, E, G)$ ▷ literal counts
  $N \leftarrow N \cup \textsc{Query}(B, Q^{cnt}_{\text{uri}}, E, G)$ ▷ untyped-URI counts
end for
$S \leftarrow \{\, (s, p, o, n_{(s,p,o)}) \mid (s, p, o) \in S \,\}$ ▷ merge; $n =$ `null` when no count retrieved
return S

function $\textsc{Query}(B, Q, E, G)$
  if $Q(B, G) = \top$ **then return** results
  end if ▷ single `SELECT DISTINCT`
  if $|B| > 1$ **then** ▷ bisect: isolates expensive classes into separate sub-queries
    return $\textsc{Query}(B_{[:m]}, Q, E, G) \cup \textsc{Query}(B_{[m:]}, Q, E, G)$
  end if
  $R \leftarrow \emptyset$; $k \leftarrow 0$
  repeat ▷ paginate single-class result set (`SELECT DISTINCT`)
    if page $Q(B, G, \ell, k) = \top$ **then**
      $R \leftarrow R \cup$ page results; $k \leftarrow k + 1$
    else
      $\ell \leftarrow \ell/2$; **retry page** k
      if $\ell < \ell_{\min}$ **then goto** *decompose*
      end if
    end if
  until empty page received
  return R

  decompose:
  if $Q = Q_{\text{to}}$ **then** ▷ predicate-first decomposition (last resort)
    $P \leftarrow \textsc{Query}(\{B[0]\}, Q_p(B[0]), E, G)$
    return $\bigcup_{p \in P} \textsc{Query}(\{B[0]\}, Q^p_{\text{to}}(B[0], p), E, G)$
  end if
  return $\emptyset$ ▷ strategies exhausted
end function

2.3 Semantic mapping layer

The mapping layer operates at four levels. Instance-based matching probes SPARQL endpoints for bioregistry [?] URI patterns to detect which datasets share entity namespaces. This requires no prior knowledge of the schema and identifies candidate mapping tar-

gets from the endpoint content directly. Curated mapping sets are imported via SSSOM sources, adding support for frameworks like SeMRA [?], covering sources such as NCIT-ChEBI. These are serialized as JSON-LD with one file per source-prefix pair. Inference then propagates new links by applying transitivity, inversion, and evidence-chain composition over the combined mapping graph as decribed in [?].

The fourth layer maps predicates to allow relaxation of expressivity for semantically related predicates, allowing for link discovery and enabling property graph exports (THIS PART IS TODO).

All mappings carry provenance metadata including evidence chains and justification predicates from the SSSOM and semapv vocabularies.

2.4 Integration of SPARQL examples

TODO, get SHACL sparql executbales from endpoints or static sites and render their diagram.

2.5 Query generator (web app)

Querying, export queried data to (graph) databases, generate scripts to reproduce the steps in Python. TODO.

2.6 Standard serializations

Extracted schemas are serialized in several formats. JSON-LD documents include a provenance block recording the extraction strategy, endpoint URL, generation timestamp, and pattern count. This makes each schema self-describing and ready for downstream use without requiring external metadata. VoID Turtle files describe class and property partitions following the extended VoID vocabulary. SHACL shapes are generated from a LinkML intermediate and can be applied directly for validation or repair. JSON-LD, SHACL and VoID turtles are compatible with standard RDF tooling including rdflib and Apache Jena. The LinkML is also used for exports such as pydantic Python APIs and SQL tables (work TODO).

Provenance blocks additionally record the tool version, Python version, duration, machine fingerprint, and, where supplied, the ORCID-bearing author list of the person or workflow that triggered the extraction. This allows mining runs to be cited and traced to specific pipeline invocations. All schema and VoID artifacts are assigned canonical

URIs under a shared base, so cross-referencing between a report, its VoID catalog, and its LinkML schema is machine-readable without extra configuration.

The web backend persists all artefact types (mined schemas, VoID catalogs, LinkML schemas, and mining reports) in a structured store, and exposes them through dedicated REST endpoints (`/api/reports`, `/api/void`, `/api/linkml`). This decouples downstream consumers from the filesystem layout of the pipeline output and allows artefacts to be queried, filtered by dataset or strategy, and downloaded in their native formats (Turtle, YAML) directly via HTTP.

2.7 Implementation and reproducibility

The methodology is packaged as two Docker images.

The *pipeline container* bundles QLever, the format converters, and the extraction and mapping tools; it is the environment used for all experiments reported in this paper. An orchestration script drives the complete workflow: building the image, running remote discovery, generating per-source QLever configurations, downloading and indexing each dataset, mining schemas locally, executing the mapping and inference stages, and collecting results. The script supports filtering to a subset of sources, skipping already-completed stages, and optional machine-level benchmarking that records wall time, CPU, and memory for each step.

A separate *web application container* packages a Flask backend with a SQLite store for the extracted schemas and mappings, together with a TypeScript frontend for interactive exploration: dataset and mapping-source selection, diagram-based schema visualization, a SHACL pattern browser and editor, and an IRI resolver-based query generator. The web container is not required to reproduce the results; it serves as an interface for navigating the outputs produced by the pipeline.

Local mining Remote SPARQL mining depends on endpoint availability, rate limits, and timeout behavior, all of which vary across engines and time. To decouple extraction from these constraints, rdf-solve supports a local mining mode that operates on downloaded data dumps.

For each source that publishes downloadable data (N-Quads, RDF/XML, or compressed archives), the pipeline generates a QLever configuration file containing the appropriate download and conversion commands. QLever [**?**] indexes the data locally and exposes a SPARQL endpoint, against which the same mining algorithm (Algorithm **??**) is applied.

QLever stores triples in six permutations with block-based compressed storage, enabling efficient scans and joins over billion-triple graphs [?]; its block-indexed `GROUP BY` and transitive property paths handle aggregation queries that cause timeouts on other engines at scale [?]. Because the local engine runs without network overhead, timeouts, or rate limits, mining completes faster and with fewer fallback activations.

The absence of network constraints makes a qualitatively different execution mode practical. For local QLever instances the pipeline first issues a *single unpaginated* `SELECT DISTINCT` query that attempts to retrieve all typed-object patterns from the full index in one request. If this one-shot query succeeds, which it reliably does for all but the very largest datasets, no fallback logic is needed and the schema is complete in a single trip. Only when the one-shot query fails (e.g. due to an out-of-memory condition on an unusually large dataset) does the pipeline activate the multi-level fallback chain described in Algorithm **??**: class-level bisection, adaptive pagination, and predicate-first decomposition. This execution path is *only* available for local mining; remote endpoints always begin directly with the fallback chain, since the endpoint's timeout and rate-limit behaviour is unknown.

This local mode complements remote mining. Sources that offer both a SPARQL endpoint and data dumps can be mined both ways, providing a cross-validation baseline for extraction completeness. Sources with unstable or slow endpoints can be mined locally without depending on their availability.

The pipeline supports executing the default and the untyped-as-classes extraction strategies consecutively for each source without re-downloading or re-indexing. The resulting pair of schema variants for each dataset allows downstream analysis to assess how much additional structure is recovered when untyped URI patterns are promoted to classes, and whether those candidate classes correspond to known ontology terms.

3 Results

3.1 Mining performance and reliability

We applied the mining algorithm (Section **??**) to N SPARQL endpoints, including (...).

Fallback utilization. For sources mined locally via QLever, the one-shot strategy succeeded in all cases tested, with no fallback activations required. Remote endpoint mining activated the fallback chain for a subset of sources where the direct query timed out or

returned partial results; the multi-level chain (bisection, pagination, predicate-first) recovered the patterns in each case.

Pipeline step timing. Figure **??** shows the duration of each pipeline step recorded during a full run. Total runtime includes either

- QLever indexing, one-shot schema extraction, mapping import, inference expansion, and graph construction.
- Remote endpoint mining, mapping import, inference expansion, and graph construction.

Depending on whether it was local or remote.

Figure placeholder: run `notebooks/mapping_analysis_paper.ipynb` *to generate* `figures/fig_pipeline_benchmarks.pdf`

Figure 2: Duration of each pipeline step. Generated by `notebooks/mapping_analysis_paper.ipynb`.

3.2 Pattern coverage and structural profile

Pattern-type distribution.

Figure placeholder: `figures/fig_schema_profile.pdf`

Figure 3: Distribution of pattern and class counts across mined schemas. Generated by `notebooks/mapping_analysis_paper.ipynb`.

Class coverage.

Property reuse across classes.

3.3 Mapping inventory and inference expansion

Raw mappings. Instance matching identified which endpoint pairs share URI namespaces, including Ensembl, UniProt, ChEBI, and FALDO identifiers.

Figure placeholder: `figures/fig_strategy_distribution.pdf`

Figure 4: Distribution of mapping edges by strategy (left) and predicate (right).

Imported mappings.

Inference expansion.

3.4 Cross-endpoint connectivity graph

Graph construction.

Connectivity metrics. Table **??** summarises the connectivity of the three graph variants. Each row corresponds to one variant; columns report node count, edge count, number of connected components, largest-component size, diameter, graph density, number of Louvain communities, and modularity.

Hidden bridges revealed by mappings. Dataset pairs that are reachable in G_{inferred} but have no direct edge in G_{raw} represent *hidden bridges*: connections that exist only because a transitive or inverse mapping chain links the two datasets indirectly. Figure **??** visualises how inference expansion reshapes the community structure relative to Figure **??**.

Hub ontology analysis. Datasets with the highest betweenness centrality in G_{inferred} act as integration hubs, bridging otherwise disconnected components. The heatmap in Figure **??** shows the log-scaled edge weights for the top datasets by betweenness centrality, highlighting the asymmetry between schema-level and inference-expanded connectivity.

Figure 5: Community structure of G_{schema}. Nodes = datasets; edges = shared typed-object class pairs; colours = Louvain communities.

Figure 6: Community structure of G_{inferred} after all mapping layers and inference expansion.

Connectivity visualization.

Figure 7: Log-scaled cross-dataset mapping weight for the top datasets by betweenness centrality in G_{inferred}.

3.5 Case study: cross-endpoint traversal

3.6 Comparison with prior work

Vocabulary reuse re-estimation.

Connectivity vs community detection.

3.7 Comparison with non-SPARQL-based mining methods

3.8 Schema validation and refinement

The methodology also exposes which classes are untyped in the RDF graphs (i.e., which RDF literals are used as classes by schemas), which could be used iteratively to define new ontology classes need to be added in order for the endpoint to be self-documented.

3.9 Traversal query generation and execution

4 Discussion

4.1 Work on title, why a tool on top of already powerful sparql engines

Recent engineering work on QLever addresses the *engine* side of the scalability problem: block-based storage, precomputed predicate patterns, and autocompletion over billion-triple graphs [**?**, **?**]. GRASP shows that an LLM agent can generate correct SPARQL for a single, well-maintained endpoint by probing its schema interactively [**?**]. These are substantial technical advances.

Cross-endpoint, not single-endpoint. QLever's autocompletion and GRASP's interactive probing address queries against one endpoint. Federated queries or dataset selection across the LSLOD cloud require prior knowledge of which endpoints share compatible

schemas. The connectivity graph produced here provides that prior, though its quality depends directly on the completeness of the mined schemas and the precision of the imported mapping sets, both of which carry known limitations (Section **??**).

Persistent, citable artefacts versus on-the-fly discovery. GRASP discovers schema information at query time. The artefacts produced here (VoID catalogs, SHACL shapes, LinkML schemas) are static snapshots with provenance metadata.

Open science outputs for under-resourced endpoints. The resources that QLever autocompletion and LLM approaches perform best on tend to be large, consistently available, well-maintained endpoints (Wikidata, UniProt). Much of the LSLOD cloud is: small-to-medium databases maintained by small groups, with intermittent support, no VoID-ext or other schema descriptions, and no budget for infrastructure upgrades. Publishing extracted schemas as linked data lowers the barrier for tools and workflows that cannot query the source directly. Whether this constitutes a meaningful contribution depends on adoption.

[**?**] concludes that the LSLOD cloud "is not actually as densely connected as is often indicated", with vocabulary reuse at under 20 % and many sources standing as isolated RDF conversions. Their community detection identified groups of semantically similar schema elements across sources using word-embedding similarity, revealing latent overlap invisible at the URI level. SynopsViz [**?**] and Sparklis [**?**] address exploration and querying of individual sources but do not bridge the gap across endpoints. LODsyndesis provides connectivity metrics across hundreds of datasets but operates on pre-computed indexes rather than live endpoint mining.

We materialize the connectivity that prior analyses only detected statistically. The connectivity graph (Section **??**) identifies endpoint pairs that can be concretely joined via shared ontology classes and inferred mappings, answering "which endpoints can be federated?" rather than "which schemas are similar?"

Curated mapping sets imported further cover cases where namespace alignment alone is insufficient.

4.2 Provenance and reproducibility

Each mining run produces a self-contained report that records the tool version, extraction strategy, endpoint URL, graph scope, duration, warnings found, and the identity of the

person or automated workflow that triggered it. These reports are assigned canonical URIs and cross-referenced with their corresponding VoID catalog, LinkML schema, and SHACL shapes, so the full provenance chain of any artefact can be reconstructed without consulting external logs.

Documenting the extraction strategy, including whether counts were collected and whether the untyped-as-classes variant was applied, means that two schemas generated from the same endpoint under different assumptions are distinguishable in the record. This is a prerequisite for any meaningful longitudinal comparison or method benchmarking: conclusions about schema evolution or strategy differences require that the methodological parameters of each run are unambiguously captured alongside the results.

4.3 Scalability and centralization

Papadaki et al. [**?**] highlighted the difficulty of achieving efficiency for complex aggregate queries at scale.

Kamdar and Musen [**?**] argued for centralized top-down approaches to monitor accessibility and enforce vocabulary reuse standards.

rdf-solve's multi-level fallback chain addresses the scalability concern at the extraction level for external endpoints, recovering patterns for queries tthat would otherwise fail under single-shot extraction. The local mining pipeline (Section **??**) further removes the dependency on remote endpoint availability entirely: downloading data dumps and indexing them with QLever eliminates network timeouts, rate limits, and engine-specific quirks from the extraction path. The containerized web application and batch-seeding scripts provide a reusable infrastructure for periodic re-mining, moving toward the centralized monitoring.

4.4 Mining robustness and the fallback contribution

The multi-level fallback chain (bisect, paginate, predicate-first decomposition) is an engineering contribution that enables all downstream analyses.

Prior remote endpoint miners, including the [**?**] tool, use single-shot queries per class and silently skip classes on timeout; Loupe [**?**].

The separation of pattern discovery from counting further ensures that a count failure does not cascade into a pattern loss, a design choice validated by the count-failure rate analysis.

4.5 Resulting LSLOD cloud

Schemas

Mapping layer

4.6 Ontology reuse and matching

4.7 Mapping quality and evidence chains

The mapping layer's quality is bound by its upstream sources. Instance-based matching detects shared URI namespaces but does not verify semantic equivalence; SeMRA-imported mappings [**?**] inherit the precision and recall of their curated sources. Inference propagation (transitivity, inversion) amplifies both signal and noise: transitive chains of length ≥ 3 carry multiplicative uncertainty. The evidence-chain metadata preserved in the mapping JSON-LD documents these provenance paths, but downstream consumers must decide which evidence depth to trust.

4.8 Limitations of schema mining

Graph scope and invisible types. Phase 1 enumerates only types asserted via `rdf:type` and visible under the chosen graph scope; inferred or unmaterialized types are silently absent from C. Graph selection is therefore a critical configuration choice. The whole-endpoint mining variant (no named-graph restriction) with post-mining service-namespace filtering addresses this partially: it recovers patterns that would be missed under a strict graph scope, at the cost of including infrastructure artefacts that must be stripped.

Remote mining. The remote mining step depends on endpoint availability, rate limits, and engine-specific behavior, and is inherently liable to data incompleteness. Failure risk for Q_{to} grows with instance count and object cardinality, so the output may be biased toward small, well-indexed classes. When all fallback strategies fail, a batch contributes $\emptyset$ to S, indistinguishable from a class with genuinely no patterns of that type. Failures are logged but cannot be encoded in S itself.

Different SPARQL engines (Virtuoso, Blazegraph, Fuseki, QLever) exhibit different timeout behavior, error responses... The fallback chain mitigates but does not eliminate these differences.

4.9 Broader limitations and future work

Property-level mappings. Need more.

Validation feedback loops. Feeding validation results back to dataset publishers would close the loop between schema mining and data improvement [**?**].

Downstream exports. LinkML intermediate representations open a path to SQL table generation, property-graph exports (Neo4j), and typed Python APIs (Pydantic). What else can be achieved?

References

[1] Shqiponja Ahmetaj, Iovka Boneva, Jan Hidders, Katja Hose, Maxime Jakubowski, Jose Emilio Labra Gayo, Wim Martens, Fabio Mogavero, Filip Murlak, Cem Okulmus, Axel Polleres, Ognjen Savković, Mantas Šimkus, and Dominik Tomaszuk. Common Foundations for SHACL, ShEx, and PG-Schema. In *Proceedings of the ACM on Web Conference 2025*, pages 8–21, Sydney NSW Australia, April 2025. ACM.

[2] Hannah Bast and Björn Buchhold. Qlever: A query engine for efficient sparql+ text search. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management*, pages 647–656, 2017.

[3] Nikos Bikakis, Melina Skourla, and George Papastefanatos. rdf:SynopsViz - A Framework for Hierarchical Linked Data Visual Exploration and Analysis, 2014. Version Number: 2.

[4] Jerven Bolleman. void-generator, 2015.

[5] Sijin Cheng and Olaf Hartig. FedQPL: A Language for Logical Query Plans over Heterogeneous Federations of RDF Data Sources (Extended Version), October 2020. arXiv:2010.01190 [cs].

[6] Daniel Fernández-Álvarez, Yasunori Yamamoto, Jose Emilio Labra Gayo, and Andra Waagmeester. Extracting shapes from large RDF data collections. In *Proceedings of the 15th Semantic Web Applications and Tools for Health Care and Life Sciences Conference*, 2024.

[7] Sébastien Ferré. Sparklis: An expressive query builder for SPARQL endpoints with guidance in natural language. *Semantic Web*, 8(3):405–418, December 2016.

[8] Nicola Guarino and Pierdaniele Giaretta. Ontologies and Knowledge Bases.

[9] Olaf Hartig. Zero-Knowledge Query Planning for an Iterator Implementation of Link Traversal Based Query Execution. In Grigoris Antoniou, Marko Grobelnik, Elena Simperl, Bijan Parsia, Dimitris Plexousakis, Pieter De Leenheer, and Jeff Pan, editors, *The Semantic Web: Research and Applications*, volume 6643, pages 154–169. Springer Berlin Heidelberg, Berlin, Heidelberg, 2011. Series Title: Lecture Notes in Computer Science.

[10] Charles Tapley Hoyt, Meghan Balk, Tiffany J. Callahan, Daniel Domingo-Fernández, Melissa A. Haendel, Harshad B. Hegde, Daniel S. Himmelstein, Klas Karis, John Kunze, Tiago Lubiana, Nicolas Matentzoglu, Julie McMurry, Sierra Moxon, Christopher J. Mungall, Adriano Rutz, Deepak R. Unni, Egon Willighagen, Donald Winston, and Benjamin M. Gyori. Unifying the identification of biomedical entities with the Bioregistry. *Scientific Data*, 9(1):714, November 2022.

[11] Charles Tapley Hoyt, Klas Karis, and Benjamin M Gyori. Assembly and reasoning over semantic mappings at scale for biomedical data integration. *Bioinformatics*, 41(11):btaf542, November 2025.

[12] Maulik R. Kamdar and Mark A. Musen. An empirical meta-analysis of the life sciences linked open data on the web. *Scientific Data*, 8(1):24, January 2021.

[13] Shuichi Kawashima, Toshiaki Katayama, Hideki Hatanaka, Tatsuya Kushida, and Toshihisa Takagi. NBDC RDF portal: a comprehensive repository for semantic data in life sciences. *Database*, 2018, January 2018.

[14] Pierre Maillot, Olivier Corby, Catherine Faron, Fabien Gandon, and Franck Michel. IndeGx: A model and a framework for indexing RDF knowledge graphs with SPARQL-based test suits. *Journal of Web Semantics*, 76:100775, April 2023.

[15] Pierre Maillot, Catherine Faron, Fabien Gandon, Franck Michel, and Pierre Monnin. Eat your own KR: a KR-based approach to index Semantic Web Endpoints and Knowledge Graphs, 2025. Version Number: 1.

[16] Nandana Mihindukulasooriya, Mohammad Rifat Ahmmad Rashid, Giuseppe Rizzo, Raúl García-Castro, Oscar Corcho, and Marco Torchiano. RDF shape induction using knowledge base profiling. In *Proceedings of the 33rd Annual ACM Symposium on Applied Computing*, pages 1952–1959, 2018.

[17] Maria-Evangelia Papadaki, Yannis Tzitzikas, and Michalis Mountantonakis. A Brief Survey of Methods for Analytics over RDF Knowledge Graphs. *Analytics*, 2(1):55–74, January 2023.

[18] Ruben Taelman and Ruben Verborgh. Link Traversal Query Processing Over Decentralized Environments with Structural Assumptions. In Terry R. Payne, Valentina Presutti, Guilin Qi, María Poveda-Villalón, Giorgos Stoilos, Laura Hollink, Zoi Kaoudi, Gong Cheng, and Juanzi Li, editors, *The Semantic Web – ISWC 2023*, volume 14265, pages 3–22. Springer Nature Switzerland, Cham, 2023. Series Title: Lecture Notes in Computer Science.

[19] Yael Tirlet, Jerven Bolleman, Emmanuelle Becker, Fabrice Legeai, and Olivier Dameron. Modular composition of SPARQL queries for focusing on what to look for rather than how to get it.